CSE 398/498 Lab 1 Report: Object Detection with YOLOv11 and YOLOE

Hengde Dai
hed424@lehigh.edu

September 2, 2026

1 Overview

I completed the three experiments of Lab 1: (1) ran pretrained YOLOv11 on my laptop webcam, (2) fine-tuned YOLOv11 on a 50-image custom dataset and redid the webcam test, and (3) followed the YOLOE Colab to learn open-set detection. All code, data, weights, and executed notebooks are in my GitHub repository.[1]

Environment: webcam experiments ran locally (Windows 11, RTX 3070 Ti, PyTorch 2.5.1 + CUDA 12.1); training and the YOLOE notebook ran on a Colab T4 GPU.

2 Experiment 1: Pretrained YOLOv11 on a Webcam

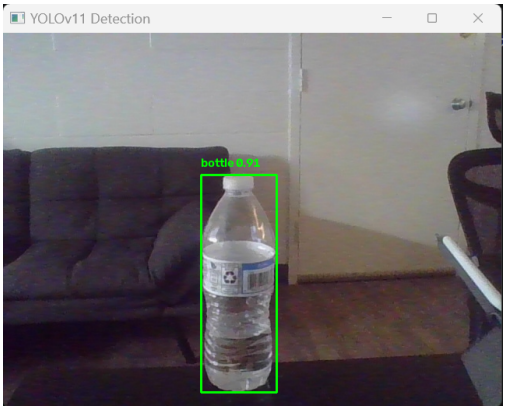
I cloned the assigned `webcam-object-recognition` repo and tested nine household objects in front of the camera. The script uses COCO-pretrained `yolo11n.pt` with a 0.5 confidence threshold. Table 1 lists the results; Figure 1 shows one success and the failure that I later used for fine-tuning.

Object	Prediction	Confidence	Outcome
Water bottle	bottle	0.91	correct
Couch	couch	0.85	correct
Scissors	scissors	0.80	correct
Book	book	0.78	correct
Cell phone	cell phone	0.78	correct
Game controller	remote	0.82	near miss (no controller class in COCO)
Vine tomatoes	apple	0.52–0.62	wrong
Toilet paper (upright)	cup	0.59	wrong
Toilet paper (on side)	toilet	0.53	wrong

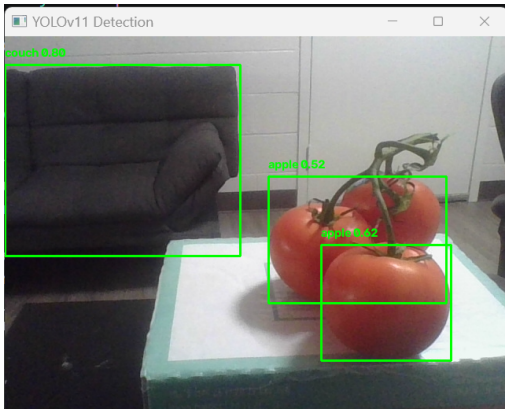
Table 1: Pretrained YOLOv11n webcam results. Screenshots are in `step1-webcam-yolov11/results-pretrained/`.

The pattern behind the failures is simple: COCO has 80 classes, and tomato and toilet paper are not among them. The model can only pick the closest class it knows, so tomatoes became "apple" and a paper roll became "cup" or "toilet."

[1] `https://github.com/DylanDai63/cse498-lab1`



(a) Correct: bottle, 0.91.



(b) Wrong: tomatoes detected as apple.

Figure 1: Pretrained YOLOv11n on the webcam.

3 How YOLOv11 Works

YOLO means "You Only Look Once": one CNN forward pass over the whole image directly outputs boxes, confidence scores, and class scores. Older two-stage detectors (e.g., R-CNN) first propose regions and then classify them; YOLO's single-stage design is less accurate but much faster, which is why the webcam demo runs in real time (about 7 ms per frame on my GPU).

Key ideas:

- **Grid prediction.** The image is divided into an $S \times S$ grid. Each cell predicts a few boxes (x, y, w, h) with confidence plus class scores. The cell containing an object's center is responsible for that object.
- **Training assignment.** Each ground-truth object is matched to the predicted box with the highest IoU, so one predictor specializes per object.
- **NMS.** At inference many overlapping boxes appear. Non-maximum suppression keeps the highest-confidence box and drops neighbors whose IoU with it exceeds a threshold.
- **Architecture.** Backbone (feature extraction) + neck (multi-scale feature fusion) + head (box/class outputs).

YOLOv11 improvements over v8 (from the course slides): the C3k2 block replaces C2f in the backbone for cheaper multi-scale features; the C2PSA attention module helps the network focus on informative regions; the head shares weights between classification and regression branches to cut latency. Net effect: higher COCO accuracy with fewer parameters. I used the nano variant `yolo11n` (2.6M parameters) in all experiments so the comparison stays fair.

4 Experiment 2: Fine-Tuning YOLOv11

4.1 Dataset (50 images, class `tomato`)

The Kaggle grocery dataset suggested in the handout turned out to be a CSV of product listings with no images, so, following the revised instructions, I built my own dataset from Google Open Images. I picked `tomato` as the target and checked programmatically that it is not among the 80 COCO class names, so it is an unseen class. The final dataset has 50 images / 99 boxes in YOLO format, split 42 train / 8 validation (Figure 2).

It took me three tries to get a dataset that works:

1. **Two classes (tomato + toilet paper):** tomato reached mAP50 0.59 but toilet paper only 0.01. Open Images has very few toilet-paper images; about 12 training images were not enough to learn a class.
2. **50 random tomato images:** mAP50 0.66, but every detection scored below 0.25 confidence. The random sample was noisy: Open Images labels sauces, slices, unripe fruit, and even a persimmon photo as "Tomato."
3. **50 hand-picked images:** I ranked candidates by box size, reviewed about 200 by hand, and kept 50 clean photos of whole, mostly red tomatoes. I also excluded dense piles where not every visible tomato is labeled, because unlabeled objects count as background during training.

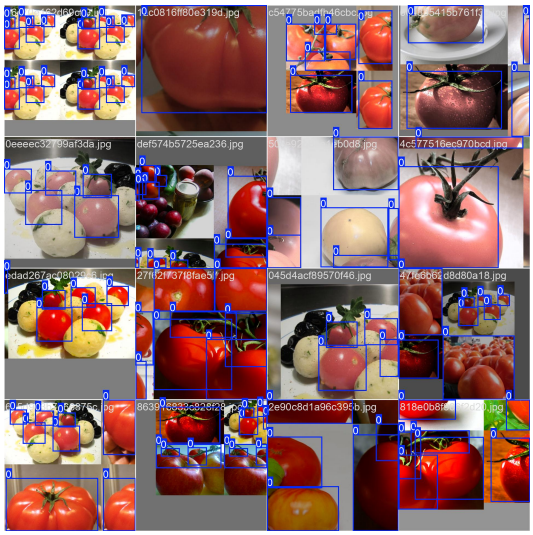


Figure 2: A training batch with Ultralytics augmentation (mosaic, flips, color jitter).

4.2 Training and Results

I followed the assigned Roboflow Colab, with three changes: model `yolo11n.pt` (to match Experiment 1), my own `data.yaml`, and `epochs=150` (a 42-image set needs many passes; an earlier 50-epoch run gave correct boxes but with confidence too low to pass normal thresholds). Training took about 5 minutes on the T4.

Results on the 8-image validation set: precision 0.788, recall 0.714, mAP50 0.727, mAP50-95 0.607. The validation set is small, so these numbers are rough indicators. Figure 3 shows the training curves; Figure 4 shows the confusion matrix (12 of 21 tomatoes detected, 9 missed, 3 false alarms) and sample predictions.

4.3 Redoing Experiment 1: Does Fine-Tuning Help?

Yes. I loaded `best.pt` in the same webcam script and held up the same vine tomatoes: the pretrained model had called them `apple` (0.62); the fine-tuned model says `tomato` at 0.96 (Figure 5). One side effect: the fine-tuned model detects nothing else. It only knows the one class it was trained on and forgot the 80 COCO classes.

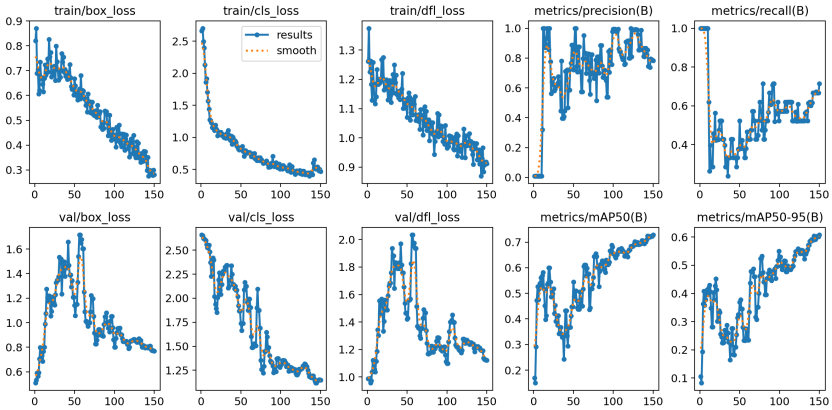
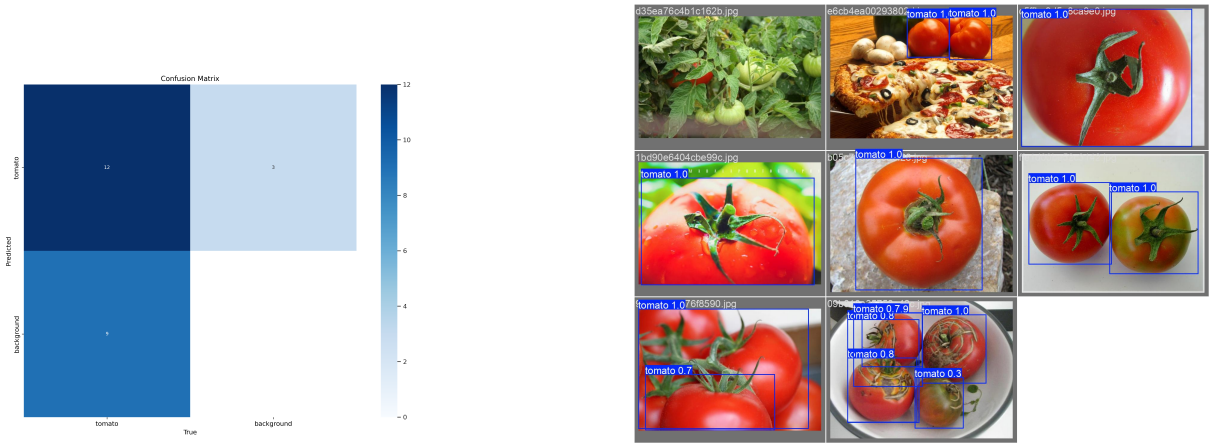


Figure 3: Losses (left) and validation metrics (right) over 150 epochs.



(a) Confusion matrix (0.25 threshold).

(b) Predictions on validation images.

Figure 4: Fine-tuned model on the validation set.

4.4 What I Learned from Fine-Tuning

- **Data quality beats quantity.** 50 noisy images gave a model that was useless in practice; 50 curated images fixed it with the same training recipe.
- **Each class needs a minimum amount of data.** Toilet paper failed with about 12 training images; tomato worked with 42. Transfer learning lowers the requirement but does not remove it.
- **Check metrics together with a threshold.** My intermediate model had decent mAP (which averages over all thresholds) yet detected nothing at 0.25. Only the confusion matrix revealed the problem.
- **Fine-tuning on new classes forgets old ones.** To keep both, one would include old classes in the training data or keep two models.

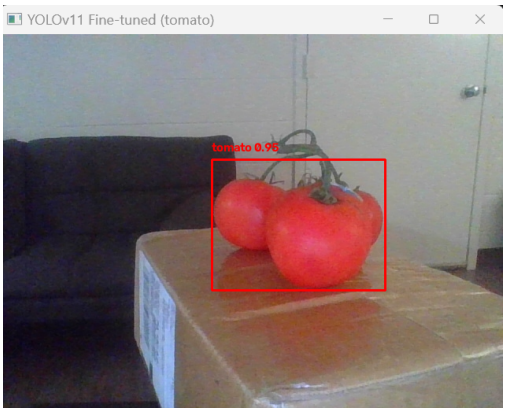


Figure 5: After fine-tuning: the same tomatoes, now `tomato` 0.96.

5 Experiment 3: Open-Set Detection with YOLOE

5.1 What Is Open-Set Object Detection?

A normal (closed-set) detector can only output the classes fixed at training time; my two models show the problem from both sides (80 classes that miss tomato, or one class that misses everything else). Open-set (open-vocabulary) detection removes the fixed list: you give the model a *prompt* at inference time, as text (class names) or as a visual example (a reference box), and it finds matching objects even for categories it was never explicitly trained on.

In the assigned YOLOE Colab, the text prompt `["dog", "eye", "nose", "tongue"]` made the model detect all four concepts on the example image, including object parts (Figure 6). No retraining was involved; changing what the model detects only takes editing the word list.

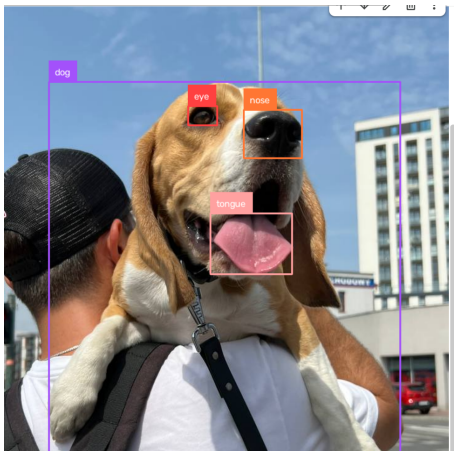


Figure 6: YOLOE text-prompt detection, zero-shot.

5.2 YOLOE Architecture and What Changed

YOLOE keeps the standard YOLO skeleton: a YOLOv8/v11-style backbone and a PAN neck that fuses the P3/P4/P5 scales, so it stays real-time. The key change is in the head:

- A closed-set YOLO head ends in a classification layer with one fixed weight row per training class.

- YOLOE replaces it with an anchor-free, decoupled head whose classification branch outputs an **embedding**, and each region is scored by **similarity against prompt embeddings**. The class list becomes an input instead of a trained parameter.
- Three prompt encoders produce the embeddings: **RepRTA** for text (class names go through a CLIP-style text encoder once, and the resulting vectors act as the classification weights), **SAVPE** for visual prompts (a reference box is encoded into the same space), and **LRPC** for prompt-free mode (a built-in vocabulary of about 4,585 classes).

Why it works: during large-scale pretraining on region-text data, the detector learns to place each region's feature close to the text embedding of its label. A new word from the user lands at a meaningful spot in the same space, so similarity matching still works. A plain COCO-trained YOLO cannot do the trick, because its features were never aligned with a text space; the alignment pretraining is exactly what YOLOE adds. Once you fix a word list, the embeddings can be folded back into a normal classification layer (re-parameterization), so deployment costs nothing extra.

Compared with Experiment 2: teaching my closed-set model one new word took a curated dataset and 150 epochs; YOLOE handles new words from one line of text, at the price of a bigger model and an expensive pretraining paid once by its authors.

AI Use Statement

I used Claude (Anthropic) as an assistant for environment setup and debugging, and for building and curating the Open Images dataset.